



UNIVERSIDAD DE COLIMA

Facultad de Ingeniería Mecánica y Eléctrica

Ingeniería en Computación Inteligente

Redes Neuronales

Moran López Luis Eduardo

Proyecto de CNN

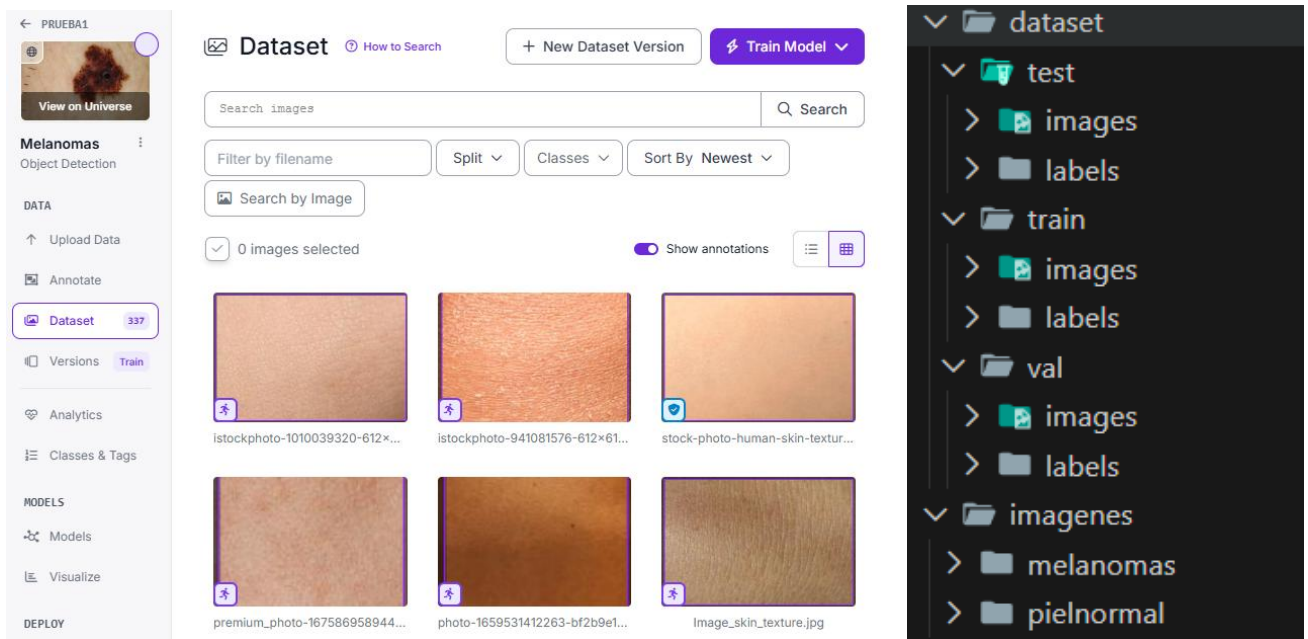
Galvez Martínez Oliver Lael

Con este proyecto de la materia elegí crear una red neuronal convolucional e implementarla al proyecto de seminario de investigación. En mi caso con mi proyecto se podía integrar, se basa en clasificar imágenes, entre melanomas y piel sana, haciendo un clasificador binario donde 0 es el melanoma y 1 la piel sana.

El sistema se divide en dos fases operacionales principales, representadas por los archivos `entrenar.py` y `prediccion.py`.

1-Crear dataset

Cree el dataset con la pagina de Roboflow, en este caso subi las imágenes y las clasifique entre train, test y validar, teniendo 70%,20% y 10% de imágenes. Despues simplemente lo descargue y ya estaban divididas las imágenes con las carpetas.



2-Cargar librerías

```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader
from torchvision import datasets, transforms
```

Configuración

device, MODEL_PATH", Define variables globales como el dispositivo (cuda o cpu), la ruta del dataset, y los hiperparámetros clave.

BATCH_SIZE = 32, se pasarán las imágenes por bloques de 32

Epochs = 10, las épocas que se utilizaras

Dispositivo, device = torch.device(...), Selecciona el hardware: Intenta usar la GPU si está disponible si no, recurre a la CPU como respaldo. En el caso de la colibeca, se usa la cpu porque no tiene grafica integrada

```
#config
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
dataset_root = "dataset"
BATCH_SIZE = 32
EPOCHS = 10
MODEL_PATH = "modelo_melanoma_pytorch.pth"
```

Transformaciones y aumento de datos

"transforms.Resize((128, 128))", Estandariza el tamaño de entrada a 128x128 pixeles.

transforms.RandomHorizontalFlip(), "Aumento de Datos: Gira la imagen horizontalmente para crear variaciones artificiales, ayudando al modelo a generalizar."

transforms.ToTensor(), Convierte la imagen a un tensor y la escala de 0-255 a 0-1.

"transforms.Normalize([0.5], [0.5])", Normalización: Centra los datos. Esto es crucial para la estabilidad y velocidad del entrenamiento.

```
# 1. TRANSFORMACIONES
train_transforms = transforms.Compose([
    transforms.Resize((128, 128)),
    transforms.RandomHorizontalFlip(),
    transforms.RandomRotation(15),
    transforms.ToTensor(),
    transforms.Normalize([0.5, 0.5, 0.5], [0.5, 0.5, 0.5])
])

val_transforms = transforms.Compose([
    transforms.Resize((128, 128)),
    transforms.ToTensor(),
    transforms.Normalize([0.5, 0.5, 0.5], [0.5, 0.5, 0.5])
])
```

Carga de datos

datasets.ImageFolder Lee las imágenes. Asume que la estructura de carpetas (train/Melanoma, train/Normal) define las etiquetas (0, 1) automáticamente.

DataLoader, Crea un iterador que suministra los datos al modelo en lotes (`batch_size=32`).
`shuffle=True` Asegura que las imágenes se mezclen en cada época para un aprendizaje menos sesgado.

```
"print(f"Clases detectadas: ...")", "Muestra el mapeo ({'Melanoma': 0, 'Normal': 1})."
```

Arquitectura de la CNN

Capas Convolucionales, `self.conv_layers = nn.Sequential(...)`, Bloque principal de extracción de características.

Convolución, `"nn.Conv2d(3, 32, ...)"`, "Detecta patrones (bordes, texturas). El 3 es por los canales RGB; 32 es el número de filtros a buscar."

Activación, `nn.ReLU()`, Introduce no linealidad al modelo, permitiéndole aprender relaciones complejas.

Pooling, `"nn.MaxPool2d(2, 2)"`, Reduce las dimensiones espaciales a la mitad. Esto hace que el modelo sea más robusto a pequeños desplazamientos.

Capas Densa, `self.fc_layers = nn.Sequential(...)`, Bloque de clasificación que toma las características extraídas para dar una predicción.

Aplanamiento, `nn.Flatten()`, Convierte la matriz 3D de características en un vector 1D ($128 \times 16 \times 16$ a 32768 elementos).

Capas Lineales, `nn.Linear(...)`, "Capas totalmente conectadas. La última (`nn.Linear(128, 1)`) da la salida final (un logit)."

Regularización, `nn.Dropout(0.5)`, Desactiva el 50% de las neuronas al azar en cada paso de entrenamiento para evitar el sobreajuste.

Flujo, `"def forward (self, x):"`, Define el orden en que los datos viajan a través de las capas del modelo.

Pérdida, `nn.BCEWithLogitsLoss()`, Mide el "error" del modelo. Es la función de pérdida más estable para problemas binarios en PyTorch.

Optimizador, `"optim.Adam(model.parameters)"`, El algoritmo que ajusta los pesos del modelo después de cada lote para reducir la pérdida.

Backpropagation, `loss.backward()`, Calcula cómo afecta el error a cada peso del modelo.

Actualización, `optimizer.step()`, Aplica los ajustes a los pesos del modelo.

Guardar, `"torch.save(model.state_dict(), ...)"`, Almacena los pesos aprendidos del modelo en un archivo. `.pth` para su uso futuro.

Entrenamiento

Función de Pérdida, `criterion = nn.BCEWithLogitsLoss()`, Mide el error entre la predicción (outputs) y la etiqueta real (labels).

Optimizador, `optimizer = optim.Adam(...)`, Algoritmo que ajusta los pesos de la red. Adam es un optimizador eficiente que controla la magnitud de los cambios a través del `lr=0.001`.

Preparación, `optimizer.zero_grad()`, Limpia los gradientes de la iteración anterior. Este es un paso esencial y único en PyTorch.

Retropropagación, `loss.backward()`, Calcula los gradientes (derivadas) de la función de pérdida con respecto a cada peso del modelo.

Actualización, `optimizer.step()`, "Aplica los gradientes calculados para actualizar los pesos del modelo.

Guardado, `torch.save(model.state_dict(), ...)`, Después de que el entrenamiento termina, guarda los pesos aprendidos del modelo en un archivo (.pth), listos para ser usados en la predicción

```
# 4. ENTRENAMIENTO
def main():
    model = SimpleCNN().to(device)
    criterion = nn.BCEWithLogitsLoss()
    optimizer = optim.Adam(model.parameters(), lr=0.001)

    print(f"Comenzando entrenamiento en {device}...")

    loss_hist = []

    for epoch in range(EPOCHS):
        model.train()
        running_loss = 0.0

        for images, labels in train_loader:
            images, labels = images.to(device), labels.float().to(device)

            optimizer.zero_grad()
            outputs = model(images).squeeze()
            loss = criterion(outputs, labels)
            loss.backward()
            optimizer.step()

            running_loss += loss.item()

        epoch_loss = running_loss / len(train_loader)
        loss_hist.append(epoch_loss)
        print(f"Epoch {epoch+1}/{EPOCHS} - Loss: {epoch_loss:.4f}")

# 5. GUARDAR
torch.save(model.state_dict(), MODEL_PATH)
print(f"\nModelo guardado exitosamente en: {MODEL_PATH}")
```

Código de predicción

"MODEL_PATH = ""modelo_melanoma_pytorch.pth""", Define la ruta del archivo que contiene los pesos (el cerebro) del modelo entrenado previamente.

"CLASES = ['Melanoma', 'Piel Sana']", Define el orden de las etiquetas, donde índice 0 es 'Melanoma' y índice 1 es 'Piel Sana'.

```
prediccion.py > SimpleCNN > _init_
1  import torch
2  import torch.nn as nn
3  from torchvision import transforms
4  from PIL import Image
5  import os
6
7  device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
8  MODEL_PATH = "modelo_melanoma_pytorch.pth"
9
10 CLASES = ['Melanoma', 'Piel Sana']
11
```

class SimpleCNN(nn.Module):, La definición de la clase es idéntica al archivo de entrenamiento. Es obligatorio que la arquitectura esté presente para que PyTorch sepa dónde cargar los pesos del archivo. pth.

nn.Sequential(...), Contiene las capas convolucionales (nn.Conv2d), de agrupación (nn.MaxPool2d) y las capas densas (nn.Linear), manteniendo la misma forma y estructura usadas para entrenar.

model = SimpleCNN().to(device), Crea una instancia vacía de la arquitectura en el dispositivo seleccionado.

model.load_state_dict(...), Carga los pesos (parámetros) aprendidos desde el archivo. pth en la arquitectura vacía.

model.eval(), Pone el modelo en modo de evaluación. Esto es crucial para que las capas como nn.Dropout y nn.BatchNorm se comporten de forma determinista (desactivando la aleatoriedad o usando estadísticas fijas).

```
# 2. FUNCIÓN DE PREDICCIÓN
def predecir(ruta_imagen):
    if not os.path.exists(MODEL_PATH):
        print("Error: No se encuentra el archivo del modelo (.pth). Ejecuta entrenar.py primero.")
        return

    # Cargar modelo
    model = SimpleCNN().to(device)
    model.load_state_dict(torch.load(MODEL_PATH, map_location=device))
    model.eval() # Modo evaluación

    # Transformación (Solo resize, tensor y normalización)
    transform = transforms.Compose([
        transforms.Resize((128, 128)),
        transforms.ToTensor(),
        transforms.Normalize([0.5, 0.5, 0.5], [0.5, 0.5, 0.5])
    ])
    image = Image.open(ruta_imagen).convert('RGB')
    image = transform(image)
```

Procesamiento

`transform = transforms.Compose(...)`, Define las transformaciones necesarias. Estas deben ser las mismas que se usaron en la validación y test durante el entrenamiento (`Resize`, `ToTensor`, `Normalize`).

`img = Image.open(ruta_imagen).convert("RGB")`, Carga la imagen de la ruta especificada.

`img_tensor = transform(img).unsqueeze(0)`, Transforma la imagen y añade una dimensión extra (`.unsqueeze(0)`). El modelo siempre espera un lote de imágenes, por lo que esta línea convierte la imagen individual en un lote de tamaño 1.

```
# Cargar imagen
try:
    img = Image.open(ruta_imagen).convert("RGB")
    img_tensor = transform(img).unsqueeze(0).to(device)
except Exception as e:
    print(f"Error cargando la imagen: {e}")
    return
```

Inferencia y calculo

`with torch.no_grad():`, Desactiva el cálculo de gradientes. Se utiliza en inferencia y validación para ahorrar recursos y tiempo, ya que el modelo ya no necesita aprender.

`logits = model(img_tensor)`, Ejecuta la pasada hacia adelante (forward pass) para obtener la puntuación de salida (logit).

`probabilidad = torch.sigmoid(logits).item()`, Convierte el logit a probabilidad (un valor entre 0 y 1) utilizando la función Sigmoid. El resultado es el valor escalar que se utiliza para clasificar.

```
# Inferencia
with torch.no_grad():
    logits = model(img_tensor)
    probabilidad = torch.sigmoid(logits).item()

print(f"\n--- Resultado para: {ruta_imagen} ---")
print(f"Probabilidad (raw): {probabilidad:.4f}")
```

Resultados

if probabilidad > 0.5:, La probabilidad de la Clase 1 ('Piel Sana') es mayor al 50%. Se predice el índice 1.

else: (probabilidad <= 0.5), La probabilidad de la Clase 0 ('Melanoma') es mayor. Se predice el índice 0.

"print(f""Confianza: {confianza*100:.2f}%""),"Muestra el resultado final, indicando la clase predicha y el porcentaje de acierto

```
if probabilidad > 0.5:
    idx = 1
    confianza = probabilidad
else:
    idx = 0
    confianza = 1 - probabilidad

print(f"Predicción: {CLASES[idx]}")
print(f"Confianza: {confianza*100:.2f}%")
```